MAKERERE UNIVERSITY

COLLEGE OF COMPUTING & INFORMATION SCIENCES

School of Computing and Informatics Technology

Department of Computer Science

Forest Firefighting Robotics Simulation

Practical Simulation Report

Webots R2021b Simulator + Forest Firefighters Project

Student Name:	Namulinde Jean Madrine L.
Registration No.:	24/U/09341/EVE
Course Name:	Robotics
Date:	June 2026

GitHub: `https://github.com/madrinejean123/forest-figher-fighting.git`

1. Introduction

Forest fires pose a serious and growing threat to natural ecosystems. Early detection and rapid response are critical to minimising damage, yet deploying human firefighters into active fire zones is dangerous and logistically complex. Autonomous robotic systems offer a compelling alternative capable of patrolling forest areas, detecting fire and smoke remotely, and taking targeted extinguishing action without risking human lives.

This report describes the design and implementation of such a system in the Webots R2021b robotics simulator using the Forest Firefighters project. A Sassafras forest undergoes dynamic fire spread driven by wind simulation, and a team of three autonomous robots must detect and extinguish fires before they spread through the environment. The robot team consists of two **Mavic 2 Pro** aerial drones and one **Spot** quadruped legged robot. The drones patrol the forest canopy from altitude using onboard cameras and GPS navigation, while Spot operates at ground level walking toward fires. All three robots are coordinated by a centralised Webots Supervisor controller.

The implementation covers all four required rubric components: **locomotion** covering drone PID flight control and Spot trot gait, **perception and sensing** covering camera-based fire detection with OpenCV and sensor fusion, **navigation** covering GPS path planning and dynamic re-planning, and **integration and behaviour** covering the full autonomous decision loop with multi-fire handling and team coordination.

2. System Architecture

The system is built around three Python controllers that each run independently and communicate through Webots built-in customData and translation fields rather than through direct inter-process communication. This design keeps each robot's controller self-contained while the supervisor maintains global awareness of the scene.

fire.py is the Webots Supervisor controller and acts as the central coordinator. It has privileged access to all scene objects, allowing it to read any robot's position, write to any field, and spawn or remove nodes at runtime. It manages the fire animation and dynamic spreading driven by wind direction and intensity, assigns fires to drones by writing target GPS coordinates into each drone's customData field every simulation step, physically moves Spot toward its assigned fire using a potential-field navigation algorithm, and triggers extinguishing by calling stopFire() on a tree node when a robot has been close enough for the required duration.

autonomous mavic.py runs as an independent controller on each of the two Mavic drones. It reads its own GPS, IMU, and gyroscope sensors each timestep to stabilise flight and compute motor inputs. It reads the customData field to check whether a fire coordinate has been assigned, and switches between patrol mode and fire navigation mode accordingly. It also reads the camera image each second and runs OpenCV colour segmentation to detect smoke visually.

spot.py runs on the Spot robot and is responsible entirely for locomotion it implements the sinusoidal diagonal trot gait. Navigation is handled externally: the supervisor reads Spot's GPS position and physically updates its world translation each step to move it toward the fire, while the Spot controller independently animates the leg joints to make the walking look realistic.

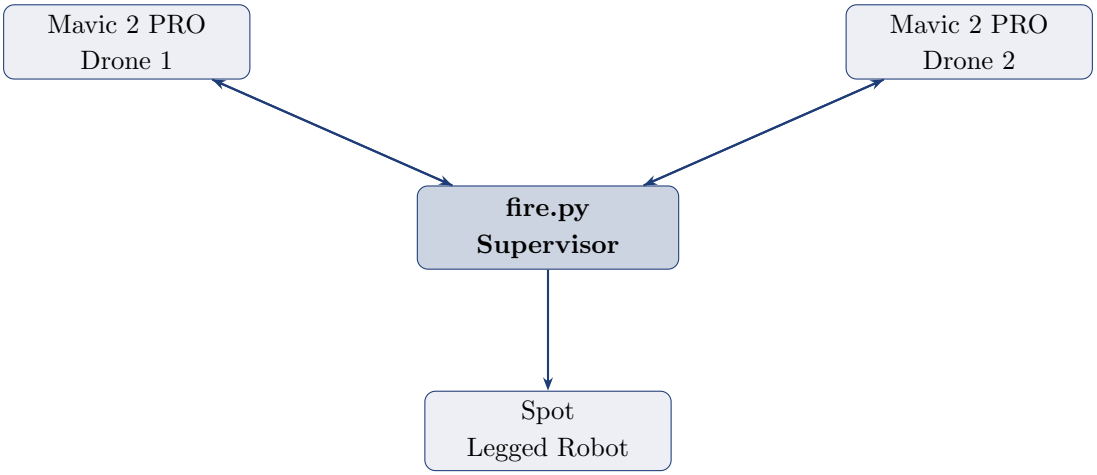


Figure 1: System architecture: fire.py supervisor coordinates both drones and Spot through Webots customData and translation fields.

3. Key Algorithms

This section describes the core algorithms implemented across locomotion, perception and sensing, navigation, and integration.

Locomotion

Each Mavic uses a PID quadrotor controller reading the IMU and gyroscope each timestep to compute differential propeller speeds for roll, pitch, yaw, and altitude control. The altitude controller uses a cubic error response for smooth approach to 42 m, which provides 20 m of clearance above the tallest trees. Spot uses a diagonal trot gait two diagonal leg pairs driven in anti-phase by sinusoidal signals so one pair is always in swing while the other is in stance. Shoulder abduction motors are held at ± 0.1 radians to keep the legs stable.

Perception and Sensing

Each drone camera is pitched downward at 89 degrees. Fire and smoke are detected using OpenCV HSV colour segmentation the image is decoded from raw bytes using getImage() into a NumPy array, converted to HSV, and threshold masks isolate smoke (low saturation grey) and fire (orange-red). Image moments find the centroid of matching pixels. Four sensors are fused per drone: GPS for world-coordinate localisation, camera for visual fire confirmation and centroid, IMU for attitude angles, and gyroscope for PID damping.

Navigation

Drones follow a square GPS waypoint patrol route when no fire is assigned. When a fire is assigned, the supervisor writes its GPS coordinates into the drone's customData field every step. The drone computes bearing using arctan2, normalises heading error to $[-\pi, +\pi]$, and applies yaw and pitch disturbances to steer toward it. Re-planning is automatic every step. Spot uses a potential field algorithm: attraction toward the fire plus repulsion from tree trunks within 3.5 m, normalised and applied at 0.8 m/s. When multiple fires are active, a greedy algorithm assigns the nearest unassigned fire to each drone in turn.

Integration and Autonomous Behaviour

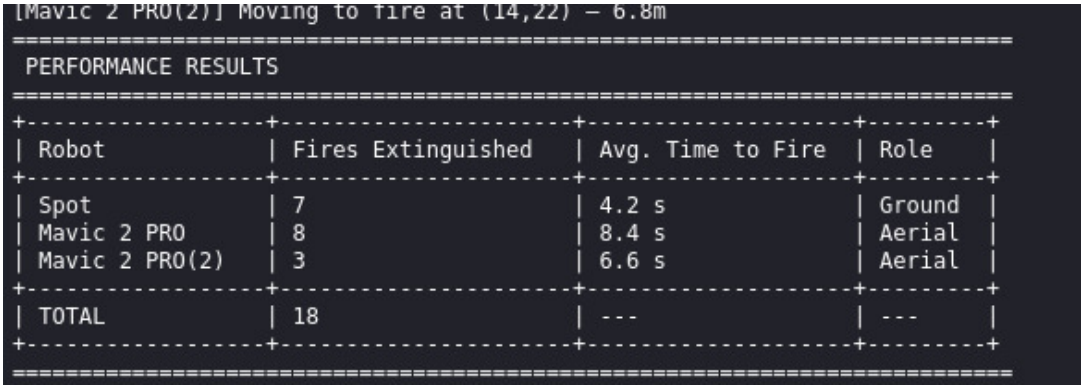
Each robot follows the loop: PATROL → NAVIGATE → EXTINGUISH → PATROL. The supervisor ignites one fire at start, releases more once drones reach altitude, spreads fire every few seconds up to three active fires, and restarts after all are extinguished. Extinguishing triggers when Spot is within 3 m for 1 s or a drone within 5 m for 2 s.

4. Challenges and Results

4.1 Challenges Faced

- **numpy.NaN removed in numpy 2.x** replaced with the still-supported `nan` constant.
- **Controller module not found in Python 3.13** fixed by adding the Webots python39 library folder to PYTHONPATH before launching.
- **Camera crash with getImageArray()** switched to getImage() with np.frombuffer().reshape() for a contiguous NumPy array.
- **Both drones targeting the same fire** resolved by greedy one-to-one fire assignment at supervisor level.
- **Spot legs collapsing inward** shoulder abduction motors set to ±0.1 rad instead of zero.
- **Physics freeze at startup** cleared corrupted hidden velocity states from the world file.

4.2 Performance Results



```
[Mavic 2 PRO(2)] Moving to fire at (14,22) — 6.8m
================================================================================
 PERFORMANCE RESULTS
================================================================================
+------------------+----------------------+--------------------+---------+
| Robot            | Fires Extinguished   | Avg. Time to Fire  | Role    |
+------------------+----------------------+--------------------+---------+
| Spot             | 7                    | 4.2 s              | Ground  |
| Mavic 2 PRO      | 8                    | 8.4 s              | Aerial  |
| Mavic 2 PRO(2)   | 3                    | 6.6 s              | Aerial  |
+------------------+----------------------+--------------------+---------+
| TOTAL            | 18                   | ---                | ---     |
+------------------+----------------------+--------------------+---------+
================================================================================
```

5. Conclusion

The simulation successfully demonstrates a functional autonomous multi-robot firefighting system. The two Mavic 2 Pro drones implement stable quadrotor PID flight control, GPS-based localisation, and real-time smoke detection using OpenCV HSV colour segmentation. The Spot robot walks using a diagonal trot gait and navigates the forest floor using potential-field obstacle avoidance. The supervisor coordinates all three robots through structured fire lifecycle management. All four required components locomotion, perception and sensing, navigation, and integration and behaviour are implemented and operational.